

# Domain adaptation

Ender Konukoglu

ETH Zürich

May 19, 2026

# Outline

## Problem with generalization

Models we train with one training set does not perform well on another data set with different, even slightly, intensity characteristics.

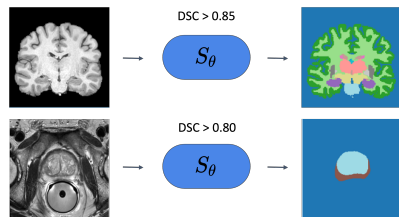
- Problem
- Domain adaptation
  - Naive approach
  - Transfer learning
  - Unsupervised domain adaptation
  - Extreme data-augmentation
  - Unsupervised source-free domain adaptation

## Section 1

### Problem

# Focus on the segmentation problem

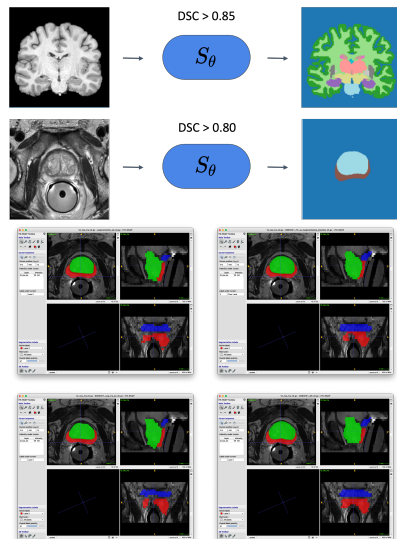
- Segmentation: pixel-wise class assignments given image intensities.
- The state-of-the-art is clearly deep learning for a variety of anatomies, pathologies and modalities
- Segmentation accuracies are close to *inter-rater variability*





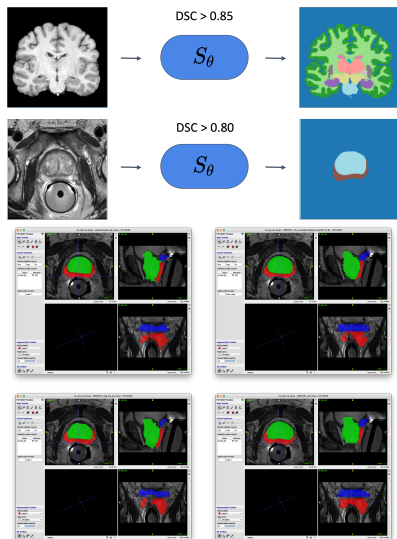
# Focus on the segmentation problem

- Segmentation: pixel-wise class assignments given image intensities.
- The state-of-the-art is clearly deep learning for a variety of anatomies, pathologies and modalities
- Segmentation accuracies are close to *inter-rater variability*

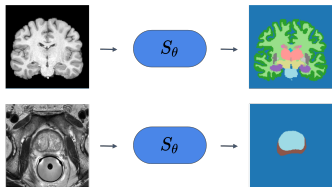


# Focus on the segmentation problem

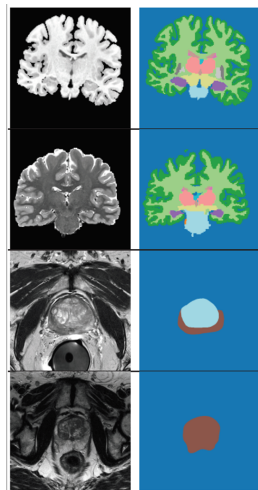
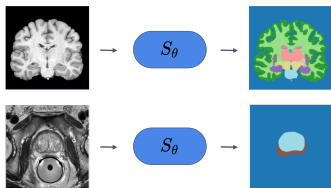
- Segmentation: pixel-wise class assignments given image intensities.
- The state-of-the-art is clearly deep learning for a variety of anatomies, pathologies and modalities
- Segmentation accuracies are close to *inter-rater variability*
- Pair-wise average DSC for whole gland prostate segmentation in a study with 6 readers and 80 patient images was 0.74.



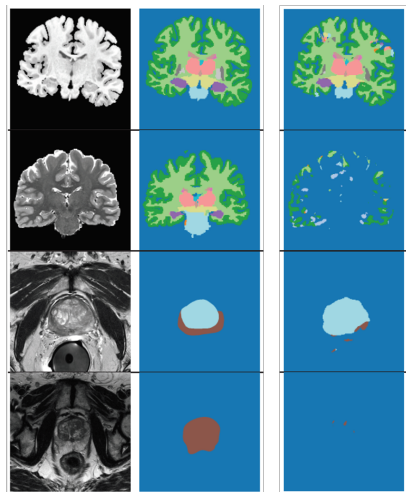
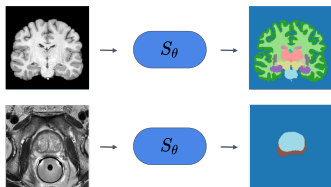
# Problems when output domain changes



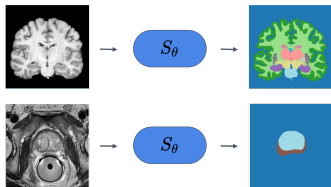
# Problems when output domain changes



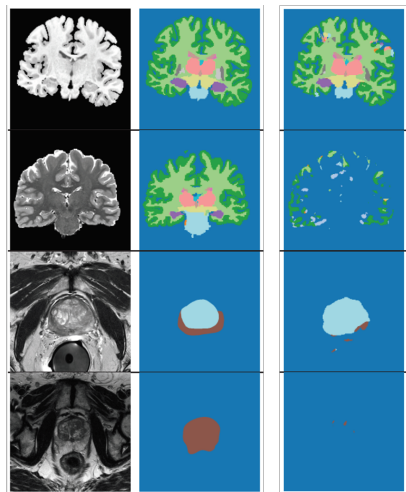
# Problems when output domain changes



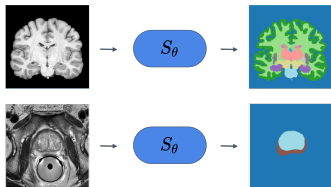
# Problems when output domain changes



Even slight differences between statistics of training and test data can lead to substantial performance degradation.

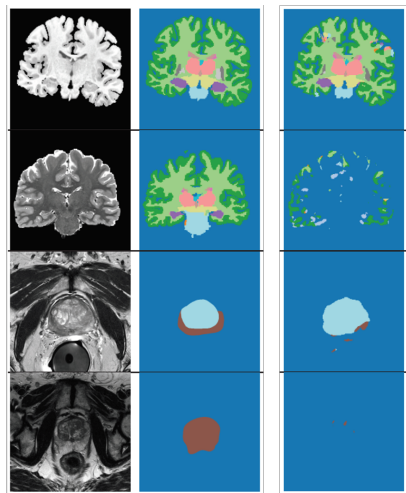


# Problems when output domain changes

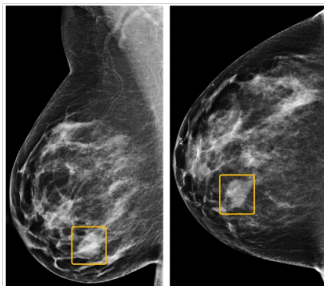


Even slight differences between statistics of training and test data can lead to substantial performance degradation.

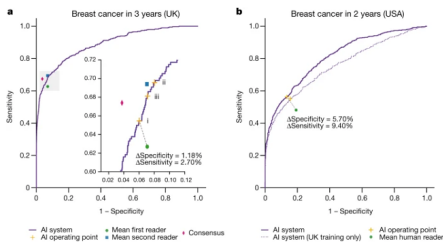
Neural networks are not robust against “domain shifts”



## Issue not specific to segmentation



**Fig. 2: Performance of the AI system and clinical readers in breast cancer prediction.**



Images from [McKinney et al. Nature 2020]

Problem is pertinent for restoration, synthesis, reconstruction, registration, ...



# Better definition of the domain shift problem

Table taken from [Castro et al. "Causality Matters in Medical Imaging", Nature Communications 2020]

| Type                | Change     | Examples of differences                                 |
|---------------------|------------|---------------------------------------------------------|
| Population shift    | $P_D(Z)$   | ages, sexes, diets, habits, ethnicities, genetics       |
| Annotation shift    | $P_D(Y X)$ | annotation policy, annotator experience                 |
| Prevalence shift    | $P_D(Y)$   | case-control balance, target selection                  |
| Manifestation shift | $P_D(Z Y)$ | anatomical manifestation of the target disease or trait |
| Acquisition shift   | $P_D(X Z)$ | scanner, resolution, contrast, modality, protocol       |

## Better definition of the domain shift problem

Table taken from [Castro et al. "Causality Matters in Medical Imaging", Nature Communications 2020]

| Type                     | Change     | Examples of differences                                  |
|--------------------------|------------|----------------------------------------------------------|
| Population shift         | $P_D(Z)$   | ages, sexes, diets, habits, ethnicities, genetics        |
| Annotation shift         | $P_D(Y X)$ | annotation policy, annotator experience                  |
| Prevalence shift         | $P_D(Y)$   | case-control balance, target selection                   |
| Manifestation shift      | $P_D(Z Y)$ | anatomical manifestation of the target disease or trait  |
| <b>Acquisition shift</b> | $P_D(X Z)$ | <b>scanner, resolution, contrast, modality, protocol</b> |

# Outline

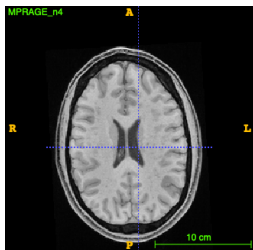
## Problem with generalization

Models we train with one training set does not perform well on another data set with different, even slightly, intensity characteristics.

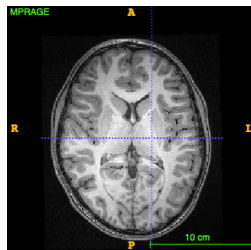
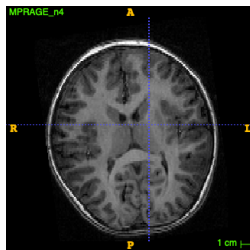
- ~~Problem~~
- Domain adaptation
  - Naive approach
  - Transfer learning
  - Unsupervised domain adaptation
  - Extreme data-augmentation
  - Unsupervised source-free domain adaptation

# Notation

**Domain:** a distinct subset of samples showing similar variations, e.g., center, scanner, sequence, acquisition protocol



Source domain samples



Target domain sample

**Source domain:** Subset of samples with which models are trained.

**Target domain:** Subset of samples on which models are applied.

Both can consist of multiple subsets, i.e., samples from multiple domains

# Domain shift

$(x^{SD}, y^{SD})$ : image and label from the source domain.

$(x^{TD}, y^{TD})$ : image and label from the target domain.

$\mathcal{D}^{SD}$ : set of images and labels from the source domain.

$\mathcal{D}^{TD}$ : set of images and labels from the target domain.

# Domain shift

$(x^{SD}, y^{SD})$ : image and label from the source domain.

$(x^{TD}, y^{TD})$ : image and label from the target domain.

$\mathcal{D}^{SD}$ : set of images and labels from the source domain.

$\mathcal{D}^{TD}$ : set of images and labels from the target domain.

Target domain  $\subset$  source domain: algorithms should generalize

# Domain shift

$(x^{SD}, y^{SD})$ : image and label from the source domain.

$(x^{TD}, y^{TD})$ : image and label from the target domain.

$\mathcal{D}^{SD}$ : set of images and labels from the source domain.

$\mathcal{D}^{TD}$ : set of images and labels from the target domain.

Target domain  $\subset$  source domain: algorithms should generalize

Target domain  $\not\subset$  source domain: **domain shift**

Training of a model happened on set of samples *different* than those seen during inference.

# Naive solution: Separate training for each domain

| At source domain                                            |                                                                    | At target domain                                            |                                                                    |
|-------------------------------------------------------------|--------------------------------------------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------|
| Training data<br>many samples<br>$\{(x_n^{SD}, y_n^{SD})\}$ | Algorithm<br>$\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta)$ | Training data<br>many samples<br>$\{(x_n^{TD}, y_n^{TD})\}$ | Algorithm<br>$\min_{\theta} \mathcal{L}(\mathcal{D}^{TD}; \theta)$ |
| Data at inference<br>$(x^{SD}, \cdot)$                      | Prediction<br>$y^{SD} \approx f(x^{SD}; \theta_{SD}^*)$            | Data at inference<br>$(x^{TD}, \cdot)$                      | Prediction<br>$y^{TD} \approx f(x^{TD}; \theta_{TD}^*)$            |

Separate training for each domain with separate training set and final model.

Inference at each domain with domain-specific model.



# Naive solution: Separate training for each domain

| At source domain                                            |                                                                    | At target domain                                            |                                                                    |
|-------------------------------------------------------------|--------------------------------------------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------|
| Training data<br>many samples<br>$\{(x_n^{SD}, y_n^{SD})\}$ | Algorithm<br>$\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta)$ | Training data<br>many samples<br>$\{(x_n^{TD}, y_n^{TD})\}$ | Algorithm<br>$\min_{\theta} \mathcal{L}(\mathcal{D}^{TD}; \theta)$ |
| Data at inference<br>$(x^{SD}, \cdot)$                      | Prediction<br>$y^{SD} \approx f(x^{SD}; \theta_{SD}^*)$            | Data at inference<br>$(x^{TD}, \cdot)$                      | Prediction<br>$y^{TD} \approx f(x^{TD}; \theta_{TD}^*)$            |

Separate training for each domain with separate training set and final model.

Inference at each domain with domain-specific model.

If enough labeled examples are present, this is the best performing model.

## Naive solution: Separate training for each domain

| At source domain                                            |                                                                    | At target domain                                            |                                                                    |
|-------------------------------------------------------------|--------------------------------------------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------|
| Training data<br>many samples<br>$\{(x_n^{SD}, y_n^{SD})\}$ | Algorithm<br>$\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta)$ | Training data<br>many samples<br>$\{(x_n^{TD}, y_n^{TD})\}$ | Algorithm<br>$\min_{\theta} \mathcal{L}(\mathcal{D}^{TD}; \theta)$ |
| Data at inference<br>$(x^{SD}, \cdot)$                      | Prediction<br>$y^{SD} \approx f(x^{SD}; \theta_{SD}^*)$            | Data at inference<br>$(x^{TD}, \cdot)$                      | Prediction<br>$y^{TD} \approx f(x^{TD}; \theta_{TD}^*)$            |

Separate training for each domain with separate training set and final model.

Inference at each domain with domain-specific model.

If enough labeled examples are present, this is the best performing model.

Requires large set of labeled samples at every domain you use the algorithm.

Labeled samples are expensive to obtain because data set curation and labeling takes a LOT of time

# Outline

## Problem with generalization

Models we train with one training set does not perform well on another data set with different, even slightly, intensity characteristics.

- ~~Problem~~
- Domain adaptation
  - ~~Naive approach~~
  - Transfer learning
  - Unsupervised domain adaptation
  - Extreme data-augmentation
  - Unsupervised source-free domain adaptation

# Transfer learning

---

**At source domain**

---

|               |           |
|---------------|-----------|
| Training data | Algorithm |
|---------------|-----------|

---

|                   |            |
|-------------------|------------|
| Data at inference | Prediction |
|-------------------|------------|

---

---

**At target domain**

---

|               |           |
|---------------|-----------|
| Training data | Algorithm |
|---------------|-----------|

---

|                   |            |
|-------------------|------------|
| Data at inference | Prediction |
|-------------------|------------|

---

# Transfer learning

## At source domain

|                                                             |                                                                    |
|-------------------------------------------------------------|--------------------------------------------------------------------|
| Training data<br>many samples<br>$\{(x_n^{SD}, y_n^{SD})\}$ | Algorithm<br>$\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta)$ |
|-------------------------------------------------------------|--------------------------------------------------------------------|

|                                        |                                                         |
|----------------------------------------|---------------------------------------------------------|
| Data at inference<br>$(x^{SD}, \cdot)$ | Prediction<br>$y^{SD} \approx f(x^{SD}; \theta_{SD}^*)$ |
|----------------------------------------|---------------------------------------------------------|

## At target domain

|               |           |
|---------------|-----------|
| Training data | Algorithm |
|---------------|-----------|

|                   |            |
|-------------------|------------|
| Data at inference | Prediction |
|-------------------|------------|

# Transfer learning

## At source domain

|                            |                                                       |
|----------------------------|-------------------------------------------------------|
| Training data              | Algorithm                                             |
| many samples               |                                                       |
| $\{(x_n^{SD}, y_n^{SD})\}$ | $\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta)$ |
| Data at inference          | Prediction                                            |
| $(x^{SD}, \cdot)$          | $y^{SD} \approx f(x^{SD}; \theta_{SD}^*)$             |

## At target domain

|                            |                                                       |
|----------------------------|-------------------------------------------------------|
| Training data              | Algorithm                                             |
| very few samples           |                                                       |
| $\{(x_n^{TD}, y_n^{TD})\}$ | $\min_{\theta} \mathcal{L}(\mathcal{D}^{TD}; \theta)$ |
| Data at inference          | Prediction                                            |

# Transfer learning

## At source domain

|                                                             |                                                                    |
|-------------------------------------------------------------|--------------------------------------------------------------------|
| Training data<br>many samples<br>$\{(x_n^{SD}, y_n^{SD})\}$ | Algorithm<br>$\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta)$ |
| Data at inference<br>$(x^{SD}, \cdot)$                      | Prediction<br>$y^{SD} \approx f(x^{SD}; \theta_{SD}^*)$            |

## At target domain

|                                                                 |                                                                    |
|-----------------------------------------------------------------|--------------------------------------------------------------------|
| Training data<br>very few samples<br>$\{(x_n^{TD}, y_n^{TD})\}$ | Algorithm<br>$\min_{\theta} \mathcal{L}(\mathcal{D}^{TD}; \theta)$ |
| Data at inference<br>$(x^{TD}, \cdot)$                          | Prediction<br>$y^{TD} \approx f(x^{TD}; \theta_{TD}^*)$            |

# Transfer learning

| At source domain                       |                                                         | At target domain                       |                                                         |
|----------------------------------------|---------------------------------------------------------|----------------------------------------|---------------------------------------------------------|
| Training data<br>many samples          | Algorithm                                               | Training data<br>very few samples      | Algorithm                                               |
| $\{(x_n^{SD}, y_n^{SD})\}$             | $\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta)$   | $\{(x_n^{TD}, y_n^{TD})\}$             | $\min_{\theta} \mathcal{L}(\mathcal{D}^{TD}; \theta)$   |
| Data at inference<br>$(x^{SD}, \cdot)$ | Prediction<br>$y^{SD} \approx f(x^{SD}; \theta_{SD}^*)$ | Data at inference<br>$(x^{TD}, \cdot)$ | Prediction<br>$y^{TD} \approx f(x^{TD}; \theta_{TD}^*)$ |

Initial training on the source domain with many samples.

Fine-tune the model at the target domain with few samples.

$$\min_{\theta} \mathcal{L}(\mathcal{D}^{TD}; \theta), \theta^{(0)} = \theta_{SD}^*$$

Model training starts at the optimal source domain parameters



# Transfer learning

| At source domain                       |                                                         | At target domain                       |                                                         |
|----------------------------------------|---------------------------------------------------------|----------------------------------------|---------------------------------------------------------|
| Training data<br>many samples          | Algorithm                                               | Training data<br>very few samples      | Algorithm                                               |
| $\{(x_n^{SD}, y_n^{SD})\}$             | $\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta)$   | $\{(x_n^{TD}, y_n^{TD})\}$             | $\min_{\theta} \mathcal{L}(\mathcal{D}^{TD}; \theta)$   |
| Data at inference<br>$(x^{SD}, \cdot)$ | Prediction<br>$y^{SD} \approx f(x^{SD}; \theta_{SD}^*)$ | Data at inference<br>$(x^{TD}, \cdot)$ | Prediction<br>$y^{TD} \approx f(x^{TD}; \theta_{TD}^*)$ |

Initial training on the source domain with many samples.

Fine-tune the model at the target domain with few samples.

$$\min_{\theta} \mathcal{L}(\mathcal{D}^{TD}; \theta), \theta^{(0)} = \theta_{SD}^*$$

Model training starts at the optimal source domain parameters

Requires only few labeled samples at the target domain.

# Transfer learning

| At source domain                       |                                                         | At target domain                       |                                                         |
|----------------------------------------|---------------------------------------------------------|----------------------------------------|---------------------------------------------------------|
| Training data<br>many samples          | Algorithm                                               | Training data<br>very few samples      | Algorithm                                               |
| $\{(x_n^{SD}, y_n^{SD})\}$             | $\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta)$   | $\{(x_n^{TD}, y_n^{TD})\}$             | $\min_{\theta} \mathcal{L}(\mathcal{D}^{TD}; \theta)$   |
| Data at inference<br>$(x^{SD}, \cdot)$ | Prediction<br>$y^{SD} \approx f(x^{SD}; \theta_{SD}^*)$ | Data at inference<br>$(x^{TD}, \cdot)$ | Prediction<br>$y^{TD} \approx f(x^{TD}; \theta_{TD}^*)$ |

Initial training on the source domain with many samples.

Fine-tune the model at the target domain with few samples.

$$\min_{\theta} \mathcal{L}(\mathcal{D}^{TD}; \theta), \theta^{(0)} = \theta_{SD}^*$$

Model training starts at the optimal source domain parameters

Requires only few labeled samples at the target domain.

At each target domain it still requires labeled samples for fine-tuning.

# Reducing required number of labeled samples for transfer learning

Fine-tune the model at the target domain with few samples.

$$\min_{\theta} \mathcal{L}(\mathcal{D}^{TD}; \theta), \theta^{(0)} = \theta_{SD}^*$$

How to reduce the number of needed labeled samples for the fine-tuning step?

## Reducing required number of labeled samples for transfer learning

Fine-tune the model at the target domain with few samples.

$$\min_{\theta} \mathcal{L}(\mathcal{D}^{TD}; \theta), \theta^{(0)} = \theta_{SD}^*$$

How to reduce the number of needed labeled samples for the fine-tuning step?

Alternative I: Use unlabeled samples, as we have seen in the semi-supervised learning setting.

# Reducing required number of labeled samples for transfer learning

Fine-tune the model at the target domain with few samples.

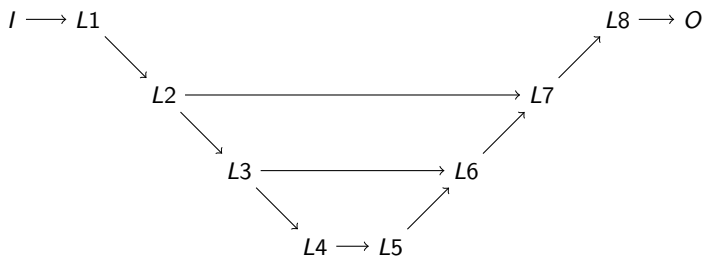
$$\min_{\theta} \mathcal{L}(\mathcal{D}^{TD}; \theta), \theta^{(0)} = \theta_{SD}^*$$

How to reduce the number of needed labeled samples for the fine-tuning step?

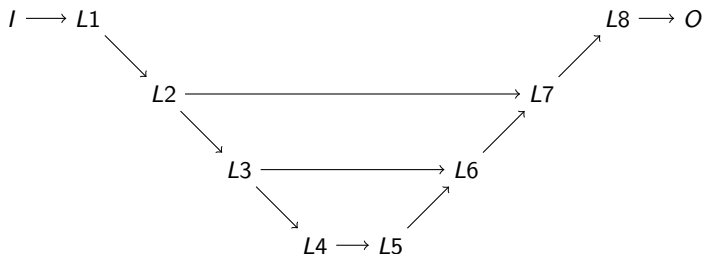
Alternative I: Use unlabeled samples, as we have seen in the semi-supervised learning setting.

Alternative II: Reduce the number of parameters to train during fine-tuning. Especially useful when unlabeled examples are not there.

# Example

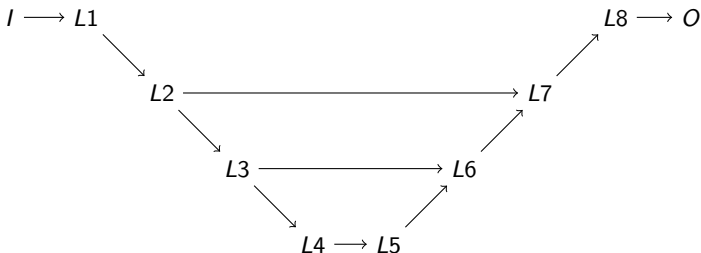


# Example



You can choose to fix most of the parameters and only change parameters between  $I$  and  $L1$ .

# Example



You can choose to fix most of the parameters and only change parameters between  $I$  and  $L1$ .

**A Lifelong Learning Approach to Brain MR Segmentation Across Scanners and Protocols**

Neerav Karani, Krishna Chaitanya, Christian Baumgartner, Ender Konukoglu

Computer Vision Lab, ETH Zurich, Zurich, Switzerland

Change only the “batch-norm” parameters in the network.

$$BN(a_I) = \gamma \frac{a_I - \mu_I}{\sqrt{\sigma_I^2 + \epsilon}} + \beta$$

All the rest remain the same.



# Fine-tuning with only 4 images

## A Lifelong Learning Approach to Brain MR Segmentation Across Scanners and Protocols

Neerav Karani, Krishna Chaitanya, Christian Baumgartner, Ender Konukoglu

Computer Vision Lab, ETH Zurich, Zurich, Switzerland

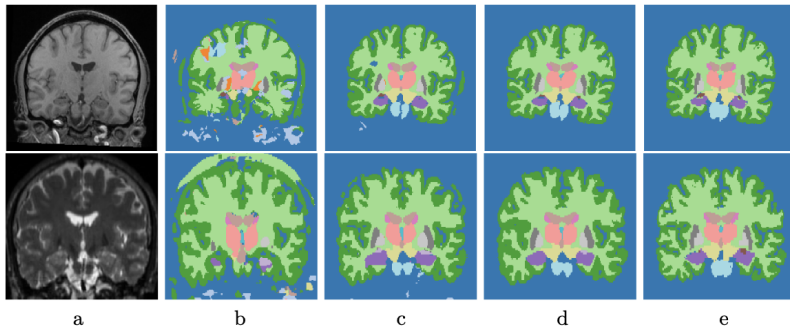


Fig. 2: Qualitative results: (a) images from domains  $D_d$ , segmentations predicted by (b)  $N_{123}^{bn}$ ,  $bn_{k^*}$ , (c)  $N_{123, k^* \rightarrow d}^{bn}$ ,  $bn_d$ , (d)  $N_d$  and (e) ground truth annotations, with  $\{d, k^*\}$  as  $\{4, 3\}$  (top) and  $\{5, 2\}$  (bottom).

# Outline

## Problem with generalization

Models we train with one training set does not perform well on another data set with different, even slightly, intensity characteristics.

- ~~Problem~~
- Domain adaptation
  - ~~Naive approach~~
  - ~~Transfer learning~~
  - Unsupervised domain adaptation
  - Extreme data-augmentation
  - Unsupervised source-free domain adaptation

# Unsupervised domain adaptation

Can we train models without any labels at the target domain?

Nothing really interesting happens at the source domain.

---

## At target domain

---

Training data

Algorithm

---

Data at inference

Prediction

---

# Unsupervised domain adaptation

Can we train models without any labels at the target domain?

Nothing really interesting happens at the source domain.

---

## At target domain

---

Training data

Algorithm

many unlabeled samples

$\{(x_n^{TD}, \cdot)\}$  and  $\{(x_n^{SD}, y_n^{SD})\}$

---

Data at inference

Prediction

---

# Unsupervised domain adaptation

Can we train models without any labels at the target domain?

Nothing really interesting happens at the source domain.

## At target domain

| Training data                                          | Algorithm                                                                                                             |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| many unlabeled samples                                 |                                                                                                                       |
| $\{(x_n^{TD}, \cdot)\}$ and $\{(x_n^{SD}, y_n^{SD})\}$ | $\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta) + \mathcal{L}_{adv}(\mathcal{D}^{TD}, \mathcal{D}^{SD}; \theta)$ |
| Data at inference                                      | Prediction                                                                                                            |

# Unsupervised domain adaptation

Can we train models without any labels at the target domain?

Nothing really interesting happens at the source domain.

## At target domain

| Training data                                                                    | Algorithm                                                                                                             |
|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| many unlabeled samples<br>$\{(x_n^{TD}, \cdot)\}$ and $\{(x_n^{SD}, y_n^{SD})\}$ | $\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta) + \mathcal{L}_{adv}(\mathcal{D}^{TD}, \mathcal{D}^{SD}; \theta)$ |
| Data at inference                                                                | Prediction                                                                                                            |
| $(x^{TD}, \cdot)$                                                                | $y^{TD} \approx f(x^{TD}; \theta_{TD}^*)$                                                                             |

# Unsupervised domain adaptation

Can we train models without any labels at the target domain?

Nothing really interesting happens at the source domain.

| At target domain                                       |                                                                                                                       |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Training data                                          | Algorithm                                                                                                             |
| many unlabeled samples                                 |                                                                                                                       |
| $\{(x_n^{TD}, \cdot)\}$ and $\{(x_n^{SD}, y_n^{SD})\}$ | $\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta) + \mathcal{L}_{adv}(\mathcal{D}^{TD}, \mathcal{D}^{SD}; \theta)$ |
| Data at inference                                      | Prediction                                                                                                            |
| $(x^{TD}, \cdot)$                                      | $y^{TD} \approx f(x^{TD}; \theta_{TD}^*)$                                                                             |

Training with source domain and unlabeled target domain samples.

$$\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta) + \mathcal{L}_{adv}(\mathcal{D}^{TD}, \mathcal{D}^{SD}; \theta)$$

An adversarial loss ensures similar features are extracted from source and target domain samples

# Unsupervised domain adaptation

Can we train models without any labels at the target domain?

Nothing really interesting happens at the source domain.

| At target domain                                       |                                                                                                                       |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Training data                                          | Algorithm                                                                                                             |
| many unlabeled samples                                 |                                                                                                                       |
| $\{(x_n^{TD}, \cdot)\}$ and $\{(x_n^{SD}, y_n^{SD})\}$ | $\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta) + \mathcal{L}_{adv}(\mathcal{D}^{TD}, \mathcal{D}^{SD}; \theta)$ |
| Data at inference                                      | Prediction                                                                                                            |
| $(x^{TD}, \cdot)$                                      | $y^{TD} \approx f(x^{TD}; \theta_{TD}^*)$                                                                             |

Training with source domain and unlabeled target domain samples.

$$\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta) + \mathcal{L}_{adv}(\mathcal{D}^{TD}, \mathcal{D}^{SD}; \theta)$$

An adversarial loss ensures similar features are extracted from source and target domain samples

Does not require any labeled images at target domain.



# Unsupervised domain adaptation

Can we train models without any labels at the target domain?

Nothing really interesting happens at the source domain.

| At target domain                                       |                                                                                                                       |
|--------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------|
| Training data                                          | Algorithm                                                                                                             |
| many unlabeled samples                                 |                                                                                                                       |
| $\{(x_n^{TD}, \cdot)\}$ and $\{(x_n^{SD}, y_n^{SD})\}$ | $\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta) + \mathcal{L}_{adv}(\mathcal{D}^{TD}, \mathcal{D}^{SD}; \theta)$ |
| Data at inference                                      | Prediction                                                                                                            |
| $(x^{TD}, \cdot)$                                      | $y^{TD} \approx f(x^{TD}; \theta_{TD}^*)$                                                                             |

Training with source domain and unlabeled target domain samples.

$$\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta) + \mathcal{L}_{adv}(\mathcal{D}^{TD}, \mathcal{D}^{SD}; \theta)$$

An adversarial loss ensures similar features are extracted from source and target domain samples

Does not require any labeled images at target domain.

A new training at each target domain and source domain images need to transported to each target domain.

# What is this adversarial loss?

## Domain-Adversarial Training of Neural Networks

Yaroslav Ganin  
Evgeniya Ustinova

GANIN@SKOLTECH.RU  
EVGENIYA.USTINOVA@SKOLTECH.RU

$$\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta) + \mathcal{L}_{adv}(\mathcal{D}^{TD}, \mathcal{D}^{SD}; \theta)$$

Goal: ensure similar features are extracted from source and target domain samples

The network is blind to differences between samples coming from the different domains.

# What is this adversarial loss?

## Domain-Adversarial Training of Neural Networks

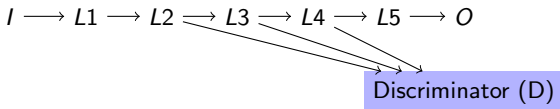
Yaroslav Ganin  
Evgeniya Ustinova

GANIN@SKOLTECH.RU  
EVGENIYA.USTINOVA@SKOLTECH.RU

$$\min_{\theta} \mathcal{L}(\mathcal{D}^{SD}; \theta) + \mathcal{L}_{adv}(\mathcal{D}^{TD}, \mathcal{D}^{SD}; \theta)$$

Goal: ensure similar features are extracted from source and target domain samples

The network is blind to differences between samples coming from the different domains.



$$\min_{\theta} \sum_n L(y_n^{SD}, f(x_n^{SD}; \theta)) + \max_{\psi} \mathbb{E}_{x \sim p_{SD}} [\log D(I(x); \psi)] \\ - \mathbb{E}_{x \sim p_{TD}} [1 - \log D(I(x); \psi)]$$

where  $I(x) = [L2(x), L3(x), L4(x)]$ .

# Distributional loss between features

$$\underbrace{\min_{\theta} \sum_n L(y_n^{SD}, f(x_n^{SD}; \theta))}_{\text{Supervised loss}} + \underbrace{\max_{\psi} \mathbb{E}_{x \sim p_{SD}} [\log D(l(x); \psi)] - \mathbb{E}_{x \sim p_{TD}} [1 - \log D(l(x); \psi)]}_{\text{Adversarial loss}}$$

Supervised loss: Task specific loss using labels on source domain samples.

Adversarial loss: Distributional loss measure distance between feature distributions of samples coming from source and target domains

You can apply to any architecture - we only looked at a very simple architecture.

## UDA at work

## Unsupervised domain adaptation in brain lesion segmentation with adversarial networks

Konstantinos Kamnitsas<sup>1,4\*</sup>, Christian Baumgartner<sup>1</sup>, Christian Ledig<sup>1</sup>,  
Virginia Newcombe<sup>2,3</sup>, Joanna Simpson<sup>2</sup>, Andrew Kane<sup>2</sup>, David Menon<sup>2,3</sup>,  
Aditya Nori<sup>4</sup>, Antonio Criminisi<sup>4</sup>, Daniel Rueckert<sup>1</sup>, and Ben Glocker<sup>1</sup>

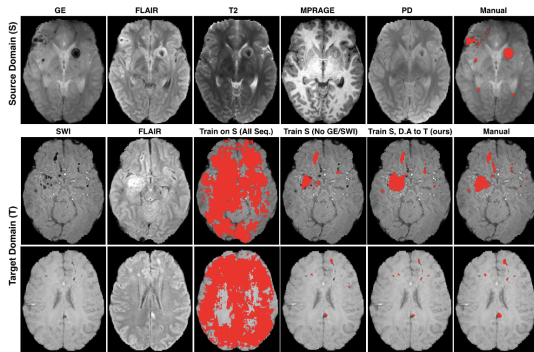


Fig. 2: (top row) Example case from  $S$ . (middle/bottom row) Visual results for two examples. A model trained on  $S$  fails on  $T$  when GE is simply replaced by SWI (3rd col.). A model trained on  $S$  using only the four common sequences misses microbleeds visible only on SWI (4th col.). Our method mitigates these problems by learning features invariant to the imaging protocol (5th col.). (T2, MPRAGE and PD of  $T$  are used but not depicted.)

## Common sides of the first three

Naive approach, Transfer learning and Unsupervised domain adaptation

- They all require training at the target domain.
- Training require some labeled data.
- Transfer learning reduces the need of labeled target domain samples of the naive approach.
- Unsupervised domain adaptation removed the need of any labeled target domain samples.
- UDA's price was to use source domain samples for every target domain training.

## Common sides of the first three

Naive approach, Transfer learning and Unsupervised domain adaptation

- They all require training at the target domain.
- Training require some labeled data.
- Transfer learning reduces the need of labeled target domain samples of the naive approach.
- Unsupervised domain adaptation removed the need of any labeled target domain samples.
- UDA's price was to use source domain samples for every target domain training.
- In the next two, we will not do training with labeled samples at the target domain.

# Outline

## Problem with generalization

Models we train with one training set does not perform well on another data set with different, even slightly, intensity characteristics.

- ~~Problem~~
- Domain adaptation
  - ~~Naive approach~~
  - ~~Transfer learning~~
  - ~~Unsupervised domain adaptation~~
  - Extreme data-augmentation
  - Unsupervised source-free domain adaptation



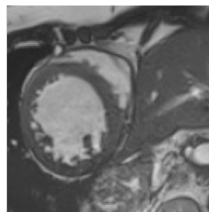
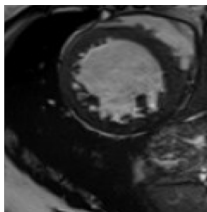
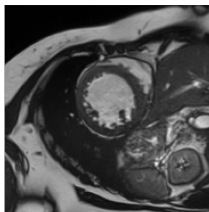
# Domain generalization framework

Domain generalization: Data augmentation, Test-time adaptation

- These methods only get trained in the source domain.
- They require no labeled samples at the target domain computations, not from target domain nor from source domain.
- At the target domain they at most do adaptation.

# Data augmentation

- Data augmentation is well established
- Transformations of the data samples serve as new samples
- Very effective way to tackle lack of data samples
- Geometric transformations - affine and elastic transformations
- Intensity transformations - blur, adding noise, gamma correction of intensities ( $I^\gamma$ )



# Data augmentation for domain generalization

---

## At source domain

---

Training data

Algorithm

---

Data at inferencePrediction

---

---

## At target domain

---

Training data

Algorithm

---

Data at inferencePrediction

---

# Data augmentation for domain generalization

## At source domain

|                                                                                                                        |                                                                                                |
|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Training data<br>many samples and augmentation<br>$\{(x_n^{SD}, y_n^{SD})\} \cup \{(\hat{x}_n^{SD}, \hat{y}_n^{SD})\}$ | Algorithm<br>$\min_{\theta} \mathcal{L}(\mathcal{D}^{SD} \cup \hat{\mathcal{D}}^{SD}; \theta)$ |
| Data at inference<br>$(x^{SD}, \cdot)$                                                                                 | Prediction<br>$y^{SD} \approx f(x^{SD}; \theta_{SD}^*)$                                        |

## At target domain

|                   |            |
|-------------------|------------|
| Training data     | Algorithm  |
| Data at inference | Prediction |

# Data augmentation for domain generalization

## At source domain

|                                                                      |                                                                                   |
|----------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Training data<br>many samples and augmentation                       | Algorithm                                                                         |
| $\{(x_n^{SD}, y_n^{SD})\} \cup \{(\hat{x}_n^{SD}, \hat{y}_n^{SD})\}$ | $\min_{\theta} \mathcal{L}(\mathcal{D}^{SD} \cup \hat{\mathcal{D}}^{SD}; \theta)$ |
| Data at inference<br>$(x^{SD}, \cdot)$                               | Prediction<br>$y^{SD} \approx f(x^{SD}; \theta_{SD}^*)$                           |

## At target domain

|                                      |            |
|--------------------------------------|------------|
| Training data<br>no training samples | Algorithm  |
| $\emptyset$                          | None       |
| Data at inference                    | Prediction |

## Data augmentation for domain generalization

**At source domain**

|                                                                      |                                                                                   |
|----------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Training data<br>many samples and augmentation                       | Algorithm                                                                         |
| $\{(x_n^{SD}, y_n^{SD})\} \cup \{(\hat{x}_n^{SD}, \hat{y}_n^{SD})\}$ | $\min_{\theta} \mathcal{L}(\mathcal{D}^{SD} \cup \hat{\mathcal{D}}^{SD}; \theta)$ |
| Data at inference<br>$(x^{SD}, \cdot)$                               | Prediction<br>$y^{SD} \approx f(x^{SD}; \theta_{SD}^*)$                           |

**At target domain**

|                                        |                                                         |
|----------------------------------------|---------------------------------------------------------|
| Training data<br>no training samples   | Algorithm                                               |
| $\emptyset$                            | None                                                    |
| Data at inference<br>$(x^{TD}, \cdot)$ | Prediction<br>$y^{TD} \approx f(x^{TD}; \theta_{SD}^*)$ |

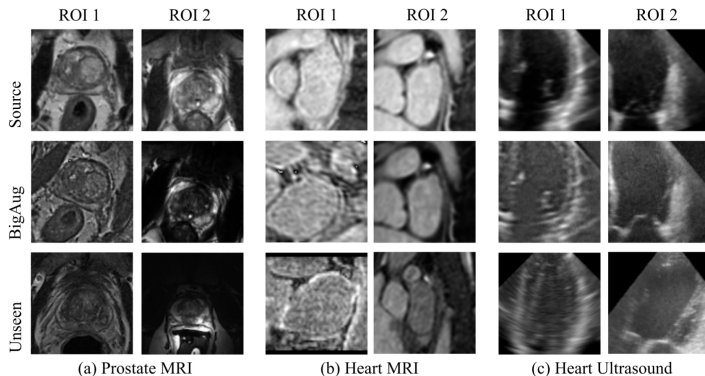
# Data augmentation for domain generalization

## Generalizing Deep Learning for Medical Image Segmentation to Unseen Domains via Deep Stacked Transformation

Ling Zhang<sup>✉</sup>, Xiaosong Wang<sup>✉</sup>, Dong Yang, Thomas Sanford, Stephanie Harmon, Baris Turkbey, Bradford J. Wood<sup>✉</sup>, Holger Roth<sup>✉</sup>, Andriy Myronenko, Daguang Xu, and Ziyue Xu<sup>✉</sup>

- Can be effective way to tackle lack of data samples from different domains
- The hypothesis is that augmentation can mimic new domains
- Stacked transformations:  $(\hat{x}_s, \hat{y}_s) = \tau_{p_n, m_n}^n(\tau_{p_{n-1}, m_{n-1}}^{n-1}(\cdots \tau_{p_1, m_1}^1((x_s, y_s))))$
- $\tau^n$  is the  $n^{th}$  transformation that is applied with probability  $p_n$  and magnitude  $m_n$ .
- Image Quality Transformations: sharpness, blurriness, noise level
- Image Appearance Transformations: brightness (intensity shifts), contrast (gamma correction)
- Spatial Configuration: rotation, scaling, deformations
- Effects of all the transformations on the labels are well known, e.g., spatial transformations transform the labels as well and appearance changes do not affect the labels.

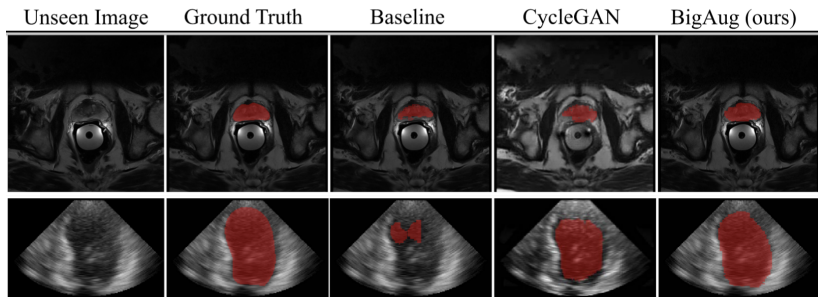
# Example augmentations



**Fig. 2.** Examples of deep stacked transformations (BigAug) results on (a) whole prostate MRI, (b) left atrial MRI, and (c) left ventricle ultrasound. 1st row: ROIs randomly cropped in volumes from source domains; 2nd row: corresponding cropped ROIs after BigAug; 3rd row: ROIs randomly cropped in volumes from unseen domains. The image pairs of 2nd–3rd rows have better visual similarity than 1st–3rd rows.

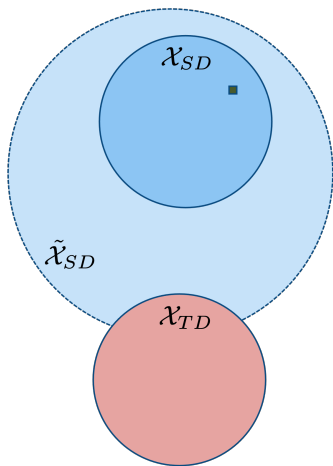


# Augmentation in action



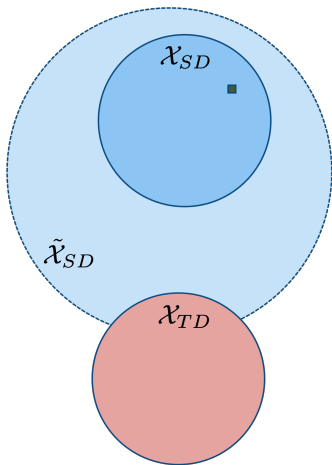
Prostate segmentation in MRI and left ventricle segmentation in ultrasound.

# Pictorially what is happening



- $\mathcal{X}_{SD}$  is the set of source domain images and  $\mathcal{X}_{TD}$  is the set of target domain images.
- Through augmentation  $\mathcal{X}_{SD}$  is expanded to  $\tilde{\mathcal{X}}_{SD}$  hoping that the expanded set overlaps with  $\mathcal{X}_{TD}$ .

## Pictorially what is happening



- $\mathcal{X}_{SD}$  is the set of source domain images and  $\mathcal{X}_{TD}$  is the set of target domain images.
- Through augmentation  $\mathcal{X}_{SD}$  is expanded to  $\tilde{\mathcal{X}}_{SD}$  hoping that the expanded set overlaps with  $\mathcal{X}_{TD}$ .
- Data augmentation provides a very strong method
- Even then, performance improvement may be limited to the variations covered by the augmentation strategy.

# Outline

## Problem with generalization

Models we train with one training set does not perform well on another data set with different, even slightly, intensity characteristics.

- ~~Problem~~
- Domain adaptation
  - ~~Naive approach~~
  - ~~Transfer learning~~
  - ~~Unsupervised domain adaptation~~
  - ~~Extreme data augmentation~~
  - Unsupervised source-free domain adaptation

# Source-free domain adaptation - test-time adaptation

---

## At source domain

---

Training data

Algorithm

---

Data at inferencePrediction

---

---

## At target domain

---

Training data

Algorithm

---

Data at inferencePrediction

---

## Source-free domain adaptation - test-time adaptation

**At source domain**

|                                                                                                                        |                                                                                                |
|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Training data<br>many samples and augmentation<br>$\{(x_n^{SD}, y_n^{SD})\} \cup \{(\hat{x}_n^{SD}, \hat{y}_n^{SD})\}$ | Algorithm<br>$\min_{\theta} \mathcal{L}(\mathcal{D}^{SD} \cup \hat{\mathcal{D}}^{SD}; \theta)$ |
| Data at inference<br>$(x^{SD}, \cdot)$                                                                                 | Prediction<br>$y^{SD} \approx f(x^{SD}; \theta_{SD}^*)$                                        |

**At target domain**

|                   |            |
|-------------------|------------|
| Training data     | Algorithm  |
| Data at inference | Prediction |

## Source-free domain adaptation - test-time adaptation

**At source domain**

|                                                                      |                                                                                   |
|----------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| Training data<br>many samples and augmentation                       | Algorithm                                                                         |
| $\{(x_n^{SD}, y_n^{SD})\} \cup \{(\hat{x}_n^{SD}, \hat{y}_n^{SD})\}$ | $\min_{\theta} \mathcal{L}(\mathcal{D}^{SD} \cup \hat{\mathcal{D}}^{SD}; \theta)$ |
| Data at inference<br>$(x^{SD}, \cdot)$                               | Prediction<br>$y^{SD} \approx f(x^{SD}; \theta_{SD}^*)$                           |

**At target domain**

|                                      |            |
|--------------------------------------|------------|
| Training data<br>no training samples | Algorithm  |
| $\emptyset$                          | None       |
| Data at inference                    | Prediction |

## Source-free domain adaptation - test-time adaptation

**At source domain**

|                                                                                                                        |                                                                                                |
|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Training data<br>many samples and augmentation<br>$\{(x_n^{SD}, y_n^{SD})\} \cup \{(\hat{x}_n^{SD}, \hat{y}_n^{SD})\}$ | Algorithm<br>$\min_{\theta} \mathcal{L}(\mathcal{D}^{SD} \cup \hat{\mathcal{D}}^{SD}; \theta)$ |
| Data at inference<br>$(x^{SD}, \cdot)$                                                                                 | Prediction<br>$y^{SD} \approx f(x^{SD}; \theta_{SD}^*)$                                        |

**At target domain**

|                                                     |                                                                                                                                |
|-----------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Training data<br>no training samples<br>$\emptyset$ | Algorithm<br>None                                                                                                              |
| Data at inference<br>$(x^{TD}, \cdot)$              | Prediction<br>$\min_{\phi} \mathcal{L}(f(x^{TD}; \theta_{SD}^*, \phi))$ then $y^{TD} \approx f(x^{TD}; \theta_{SD}^*, \phi^*)$ |



## Two main questions in test-time adaptation

- 1 What cost function will you optimize?

$$\mathcal{L}(f(x^{TD}; \theta_{SD}^*, \phi)) = ?$$

- 2 Given a network and a cost function, which parameters will you train / fine-tune?

$$f(x; \theta, \phi)$$

What is  $\phi$ ?

# What cost function?

# What cost function?

- Entropy of the prediction [Wang et al. ICLR 2021]: (for classification)  
$$H(f(x; \theta)) = - \sum_k f_k(x; \theta) \log(f_k(x; \theta))$$

Assumption is that pushing confidence of the predictions higher leads to good domain generalization.

# What cost function?

- Entropy of the prediction [Wang et al. ICLR 2021]: (for classification)  
$$H(f(x; \theta)) = - \sum_k f_k(x; \theta) \log(f_k(x; \theta))$$

Assumption is that pushing confidence of the predictions higher leads to good domain generalization.

Simple to implement, no prior information, applicable to different tasks.

# What cost function?

- Entropy of the prediction [Wang et al. ICLR 2021]: (for classification)

$$H(f(x; \theta)) = - \sum_k f_k(x; \theta) \log(f_k(x; \theta))$$

Assumption is that pushing confidence of the predictions higher leads to good domain generalization.

Simple to implement, no prior information, applicable to different tasks.

Strong assumption. Holds if the prediction is not very off and if you have many test samples that you adapt to all together.

# What cost function?

- Entropy of the prediction [Wang et al. ICLR 2021]: (for classification)  
$$H(f(x; \theta)) = - \sum_k f_k(x; \theta) \log(f_k(x; \theta))$$

Assumption is that pushing confidence of the predictions higher leads to good domain generalization.

Simple to implement, no prior information, applicable to different tasks.

Strong assumption. Holds if the prediction is not very off and if you have many test samples that you adapt to all together.
- Auto-encoder loss of input, output and intermediate features [He et al. MICCAI 2020]  
Auto-encoders are trained with source domain. Differences between their inputs and outputs are evaluated as the cost.

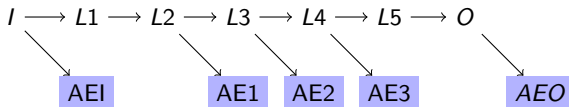
## Auto-encoder-based test time adaptation

An auto-encoder (AE) is a map from an image to the image itself:

$x \approx D(E(x))$  with an encoder  $E$  and a decoder  $D$ .

The AE gives a good reconstruction on samples coming from the domain it was trained for.

The idea here is that if we can change the parameters of the network, such that input, intermediate features and output can be reconstructed accurately by AEs trained on the source domain, that means they are *converted* to source domain-like images, and network will do accurate predictions.



# What cost function?

- Entropy of the prediction [Wang et al. ICLR 2021]: (for classification)  
$$H(f(x; \theta)) = - \sum_k f_k(x; \theta) \log(f_k(x; \theta))$$

Assumption is that pushing confidence of the predictions higher leads to good domain generalization.

Simple to implement, no prior information, applicable to different tasks.

Strong assumption. Holds if the prediction is not very off and if you have many test samples that you adapt to all together.
- Auto-encoder loss of input, output and intermediate features [He et al. MICCAI 2020]  
Auto-encoders are trained with source domain. Differences between their inputs and outputs are evaluated as the cost.



# What cost function?

- Entropy of the prediction [Wang et al. ICLR 2021]: (for classification)  
 $H(f(x; \theta)) = - \sum_k f_k(x; \theta) \log(f_k(x; \theta))$   
Assumption is that pushing confidence of the predictions higher leads to good domain generalization.  
Simple to implement, no prior information, applicable to different tasks.  
Strong assumption. Holds if the prediction is not very off and if you have many test samples that you adapt to all together.
- Auto-encoder loss of input, output and intermediate features [He et al. MICCAI 2020]  
Auto-encoders are trained with source domain. Differences between their inputs and outputs are evaluated as the cost.  
Generally applicable to different tasks and strong prior model

# What cost function?

- Entropy of the prediction [Wang et al. ICLR 2021]: (for classification)

$$H(f(x; \theta)) = - \sum_k f_k(x; \theta) \log(f_k(x; \theta))$$

Assumption is that pushing confidence of the predictions higher leads to good domain generalization.

Simple to implement, no prior information, applicable to different tasks.

Strong assumption. Holds if the prediction is not very off and if you have many test samples that you adapt to all together.

- Auto-encoder loss of input, output and intermediate features [He et al. MICCAI 2020]

Auto-encoders are trained with source domain. Differences between their inputs and outputs are evaluated as the cost.

Generally applicable to different tasks and strong prior model

Input and intermediate features can be prone to domain shifts themselves

# What cost function?

- Entropy of the prediction [Wang et al. ICLR 2021]: (for classification)  
 $H(f(x; \theta)) = - \sum_k f_k(x; \theta) \log(f_k(x; \theta))$   
Assumption is that pushing confidence of the predictions higher leads to good domain generalization.  
Simple to implement, no prior information, applicable to different tasks.  
Strong assumption. Holds if the prediction is not very off and if you have many test samples that you adapt to all together.
- Auto-encoder loss of input, output and intermediate features [He et al. MICCAI 2020]  
Auto-encoders are trained with source domain. Differences between their inputs and outputs are evaluated as the cost.  
Generally applicable to different tasks and strong prior model  
Input and intermediate features can be prone to domain shifts themselves
- Segmentation prior evaluated at the output assessing plausibility of the result [Karani et al. Media 2021]

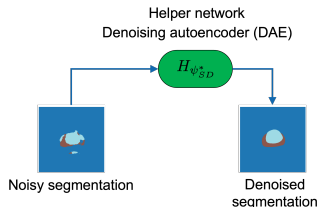
## Segmentation prior-based test time adaptation

A denoising auto-encoder (DAE) is a map from a noisy version of an image to itself  $x \approx D(E(\tilde{x}))$  with an encoder  $E$  and a decoder  $D$ .

The DAE is trained on segmentation maps to learn a prior model on the segmentations.

The DAE gives a good reconstruction on samples coming from the domain it was trained for.

The idea here is that if we can change the parameters of the network, such that the output can be reconstructed accurately by DAE trained on the source domain, that means they are *converted* to source domain-like outputs, and network will do accurate predictions.



$$\mathcal{L}(f(x; \theta, \phi), H_{\psi_{SD}^*}(f(x; \theta, \phi)))$$

## What cost function?

- Entropy of the prediction [Wang et al. ICLR 2021]: (for classification)  
 $H(f(x; \theta)) = - \sum_k f_k(x; \theta) \log(f_k(x; \theta))$   
Assumption is that pushing confidence of the predictions higher leads to good domain generalization.  
Simple to implement, no prior information, applicable to different tasks.  
Strong assumption. Holds if the prediction is not very off and if you have many test samples that you adapt to all together.
- Auto-encoder loss of input, output and intermediate features [He et al. MICCAI 2020]  
Auto-encoders are trained with source domain. Differences between their inputs and outputs are evaluated as the cost.  
Generally applicable to different tasks and strong prior model  
Input and intermediate features can be prone to domain shifts themselves
- Segmentation prior evaluated at the output assessing plausibility of the result [Karani et al. Media 2021]

## What cost function?

- Entropy of the prediction [Wang et al. ICLR 2021]: (for classification)  
 $H(f(x; \theta)) = - \sum_k f_k(x; \theta) \log(f_k(x; \theta))$   
Assumption is that pushing confidence of the predictions higher leads to good domain generalization.  
Simple to implement, no prior information, applicable to different tasks.  
Strong assumption. Holds if the prediction is not very off and if you have many test samples that you adapt to all together.
- Auto-encoder loss of input, output and intermediate features [He et al. MICCAI 2020]  
Auto-encoders are trained with source domain. Differences between their inputs and outputs are evaluated as the cost.  
Generally applicable to different tasks and strong prior model  
Input and intermediate features can be prone to domain shifts themselves
- Segmentation prior evaluated at the output assessing plausibility of the result [Karani et al. Media 2021]  
Strong prior leads to accurate segmentation results. Segmentations are not affected by intensity changes so the model is not affected by domain shifts of this sort.

## What cost function?

- Entropy of the prediction [Wang et al. ICLR 2021]: (for classification)

$$H(f(x; \theta)) = - \sum_k f_k(x; \theta) \log(f_k(x; \theta))$$

Assumption is that pushing confidence of the predictions higher leads to good domain generalization.

Simple to implement, no prior information, applicable to different tasks.

Strong assumption. Holds if the prediction is not very off and if you have many test samples that you adapt to all together.

- Auto-encoder loss of input, output and intermediate features [He et al. MICCAI 2020]

Auto-encoders are trained with source domain. Differences between their inputs and outputs are evaluated as the cost.

Generally applicable to different tasks and strong prior model

Input and intermediate features can be prone to domain shifts themselves

- Segmentation prior evaluated at the output assessing plausibility of the result [Karani et al. Media 2021]

Strong prior leads to accurate segmentation results. Segmentations are not affected by intensity changes so the model is not affected by domain shifts of this sort.

Specific to segmentation task

Which parameters will you train?



## Which parameters will you train?

- Train / fine-tune all the parameters, i.e.,  $\phi = \theta$ .

## Which parameters will you train?

- Train / fine-tune all the parameters, i.e.,  $\phi = \theta$ .  
Model will be very flexible

## Which parameters will you train?

- Train / fine-tune all the parameters, i.e.,  $\phi = \theta$ .

Model will be very flexible

Too much flexibility can lead to a result that perfectly satisfies the cost but the output is no longer related to the input

## Which parameters will you train?

- Train / fine-tune all the parameters, i.e.,  $\phi = \theta$ .

Model will be very flexible

Too much flexibility can lead to a result that perfectly satisfies the cost but the output is no longer related to the input

- Train / fine-tune a part of the parameters, i.e.,  $\phi \subset \theta$ .

## Which parameters will you train?

- Train / fine-tune all the parameters, i.e.,  $\phi = \theta$ .  
Model will be very flexible  
Too much flexibility can lead to a result that perfectly satisfies the cost but the output is no longer related to the input
- Train / fine-tune a part of the parameters, i.e.,  $\phi \subset \theta$ .  
Model will be less flexible so the issue with too much flexibility will not be an issue

## Which parameters will you train?

- Train / fine-tune all the parameters, i.e.,  $\phi = \theta$ .

Model will be very flexible

Too much flexibility can lead to a result that perfectly satisfies the cost but the output is no longer related to the input

- Train / fine-tune a part of the parameters, i.e.,  $\phi \subset \theta$ .

Model will be less flexible so the issue with too much flexibility will not be an issue

Initial model parameters are obtained using large curated data sets. It is desirable not to change them if not needed. It was costly to train them in terms of time, data and money.

## Which parameters will you train?

- Train / fine-tune all the parameters, i.e.,  $\phi = \theta$ .  
Model will be very flexible  
Too much flexibility can lead to a result that perfectly satisfies the cost but the output is no longer related to the input
- Train / fine-tune a part of the parameters, i.e.,  $\phi \subset \theta$ .  
Model will be less flexible so the issue with too much flexibility will not be an issue  
Initial model parameters are obtained using large curated data sets. It is desirable not to change them if not needed. It was costly to train them in terms of time, data and money.
- Train / fine-tune batch-norm parameters

## Which parameters will you train?

- Train / fine-tune all the parameters, i.e.,  $\phi = \theta$ .  
Model will be very flexible  
Too much flexibility can lead to a result that perfectly satisfies the cost but the output is no longer related to the input
- Train / fine-tune a part of the parameters, i.e.,  $\phi \subset \theta$ .  
Model will be less flexible so the issue with too much flexibility will not be an issue  
Initial model parameters are obtained using large curated data sets. It is desirable not to change them if not needed. It was costly to train them in terms of time, data and money.
- Train / fine-tune batch-norm parameters  
Very simple application and very few parameters.



## Which parameters will you train?

- Train / fine-tune all the parameters, i.e.,  $\phi = \theta$ .  
Model will be very flexible  
Too much flexibility can lead to a result that perfectly satisfies the cost but the output is no longer related to the input
- Train / fine-tune a part of the parameters, i.e.,  $\phi \subset \theta$ .  
Model will be less flexible so the issue with too much flexibility will not be an issue  
Initial model parameters are obtained using large curated data sets. It is desirable not to change them if not needed. It was costly to train them in terms of time, data and money.
- Train / fine-tune batch-norm parameters  
Very simple application and very few parameters.  
Batch norm parameters in deeper layers can affect results a lot and it is unclear if they address changes in intensity characteristics.

## Which parameters will you train?

- Train / fine-tune all the parameters, i.e.,  $\phi = \theta$ .  
Model will be very flexible  
Too much flexibility can lead to a result that perfectly satisfies the cost but the output is no longer related to the input
- Train / fine-tune a part of the parameters, i.e.,  $\phi \subset \theta$ .  
Model will be less flexible so the issue with too much flexibility will not be an issue  
Initial model parameters are obtained using large curated data sets. It is desirable not to change them if not needed. It was costly to train them in terms of time, data and money.
- Train / fine-tune batch-norm parameters  
Very simple application and very few parameters.  
Batch norm parameters in deeper layers can affect results a lot and it is unclear if they address changes in intensity characteristics.
- Train / fine-tune a shallow “adaptation” module prepended to the network, i.e.,  $f(x; \theta, \phi) = f(N(x; \phi); \theta)$

## Which parameters will you train?

- Train / fine-tune all the parameters, i.e.,  $\phi = \theta$ .  
Model will be very flexible  
Too much flexibility can lead to a result that perfectly satisfies the cost but the output is no longer related to the input
- Train / fine-tune a part of the parameters, i.e.,  $\phi \subset \theta$ .  
Model will be less flexible so the issue with too much flexibility will not be an issue  
Initial model parameters are obtained using large curated data sets. It is desirable not to change them if not needed. It was costly to train them in terms of time, data and money.
- Train / fine-tune batch-norm parameters  
Very simple application and very few parameters.  
Batch norm parameters in deeper layers can affect results a lot and it is unclear if they address changes in intensity characteristics.
- Train / fine-tune a shallow “adaptation” module prepended to the network, i.e.,  $f(x; \theta, \phi) = f(N(x; \phi); \theta)$   
Shallow module will not be too flexible, and we do not modify pre-trained parameters.

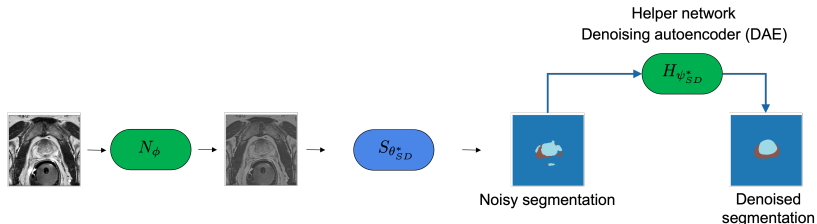
## Which parameters will you train?

- Train / fine-tune all the parameters, i.e.,  $\phi = \theta$ .  
Model will be very flexible  
Too much flexibility can lead to a result that perfectly satisfies the cost but the output is no longer related to the input
- Train / fine-tune a part of the parameters, i.e.,  $\phi \subset \theta$ .  
Model will be less flexible so the issue with too much flexibility will not be an issue  
Initial model parameters are obtained using large curated data sets. It is desirable not to change them if not needed. It was costly to train them in terms of time, data and money.
- Train / fine-tune batch-norm parameters  
Very simple application and very few parameters.  
Batch norm parameters in deeper layers can affect results a lot and it is unclear if they address changes in intensity characteristics.
- Train / fine-tune a shallow “adaptation” module prepended to the network, i.e.,  $f(x; \theta, \phi) = f(N(x; \phi); \theta)$   
Shallow module will not be too flexible, and we do not modify pre-trained parameters.  
We make the network slightly bigger

# A complete TTA model

## Test-time adaptable neural networks for robust medical image segmentation

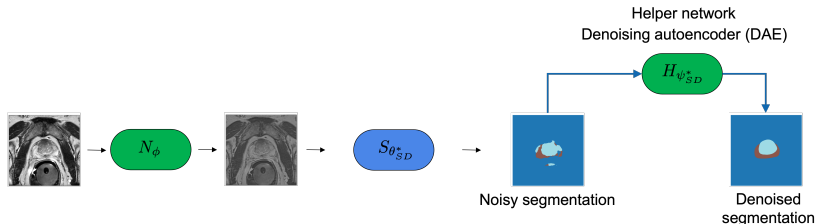
Neerav Karani , Ertunc Erdil, Krishna Chaitanya, Ender Konukoglu



$$\min_{\phi} L(\tilde{Y}, H_{\psi_{SD}^*}(\tilde{Y})) = \min_{\phi} L(S_{\theta_{SD}^*} \circ N_{\phi}(I), H_{\psi_{SD}^*}(S_{\theta_{SD}^*} \circ N_{\phi}(I)))$$

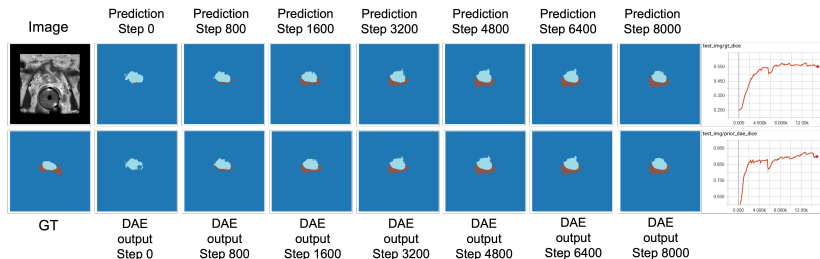
$$\tilde{Y} = S_{\theta_{SD}^*} \circ N_{\phi^*}(I)$$

# A complete TTA model

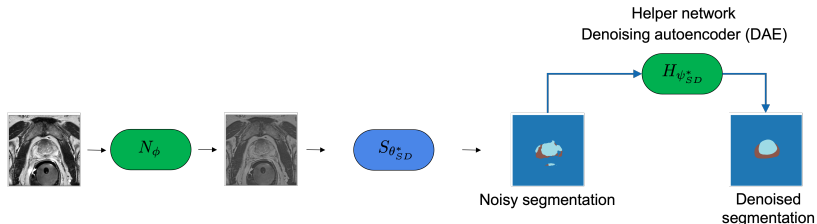


$$\min_{\phi} L(\tilde{Y}, H_{\psi^*_{SD}}(\tilde{Y})) = \min_{\phi} L(S_{\theta^*_{SD}} \circ N_{\phi}(I), H_{\psi^*_{SD}}(S_{\theta^*_{SD}} \circ N_{\phi}(I)))$$

$$\tilde{Y} = S_{\theta^*_{SD}} \circ N_{\phi^*}(I)$$

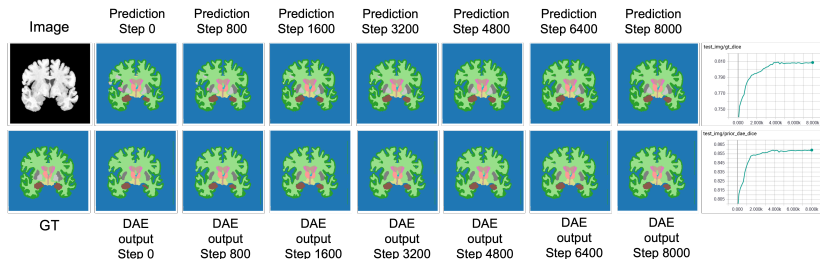


# A complete TTA model

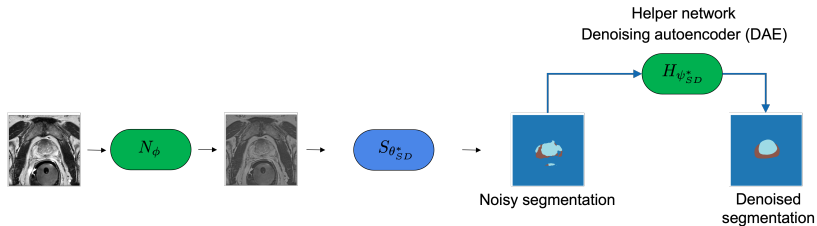


$$\min_{\phi} L(\tilde{Y}, H_{\psi^*_{SD}}(\tilde{Y})) = \min_{\phi} L(S_{\theta^*_{SD}} \circ N_{\phi}(I), H_{\psi^*_{SD}}(S_{\theta^*_{SD}} \circ N_{\phi}(I)))$$

$$\tilde{Y} = S_{\theta^*_{SD}} \circ N_{\phi^*}(I)$$

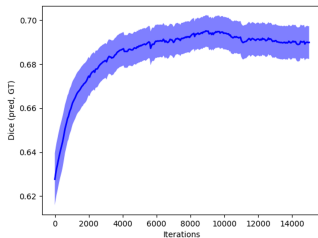
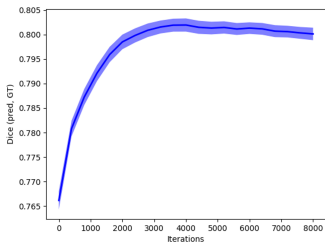


# A complete TTA model



$$\min_{\phi} L(\tilde{Y}, H_{\psi_{SD}^*}(\tilde{Y})) = \min_{\phi} L(S_{\theta_{SD}^*} \circ N_{\phi}(I), H_{\psi_{SD}^*}(S_{\theta_{SD}^*} \circ N_{\phi}(I)))$$

$$\tilde{Y} = S_{\theta_{SD}^*} \circ N_{\phi^*}(I)$$





## TTA model in action

